

02 — Agent System

Objetivo de este documento: documentar el cerebro de Puglit — el roster de 75 agentes genéticos (25 roles × 3 equipos), cómo se ensambla cada prompt, qué entra y qué no entra en la ventana de contexto, cómo se comunican y compiten los agentes, y los costos, latencias y modos de falla del sistema. Todos los nombres, stats y números acá están verificados contra el código en `/Users/alvaroz/projects/2026/puglit/web/lib/` (`roster.ts` , `playbooks.ts` , `progression.ts` , `tournament.ts` , `skill-evolution.ts` , `openai.ts` , `app-builder.ts`).

1. Agent Philosophy

Puglit no es "un LLM al que le pedís una app". Es un **sistema multi-agente genético** donde 75 identidades persistentes — cada una con un rol, una personalidad RPG y un diario de vida — compiten, se critican y aprenden entre proyectos. La tesis de diseño es simple: **un sistema que diverge, juzga y selecciona produce mejor software que un único modelo monolítico**, exactamente por la misma razón por la que la evolución produce mejores organismos que el diseño directo — diversidad + presión selectiva + herencia.

Tres ideas atraviesan todo el roster (`web/lib/roster.ts`):

1. **Las identidades son persistentes, no efímeras.** Cada agente es una fila real en `puglit_agents` con `xp` , `level` , `stats` y una reputación de calidad (`quality_sum` / `quality_n`) que se acumula a lo largo de proyectos. El comentario de cabecera del archivo lo dice literal: *"every agent is a real, persistent identity with RPG stats... Stats + level + the Queen's quality reputation accrue across projects (the agent's diary of life)."*
2. **Los stats RPG manejan los parámetros del LLM, no son cosmética.** Cada agente tiene cinco stats — `creativity` , `rigor` , `security` , `speed` , `depth` — y de ellos se deriva **determinísticamente** la temperatura de Ollama para cada llamada, vía `tempFromStats()` :

```
ts // roster.ts const t = 0.12 + s.creativity * 0.06 - s.rigor * 0.025 + s.speed * 0.005
return clamp(t, 0.1, 0.9)
```

Un rol creativo (Art Director, `creativity: 10`) corre caliente; un rol riguroso (Reliability Engineer, `rigor: 10`) corre frío y casi determinista. La personalidad **es** la configuración de inferencia.

3. **La identidad se gana, no se asigna.** A través del torneo, los agentes ganan XP, suben de nivel (curva RPG hasta nivel 1000), reciben +1 a su stat de especialidad por nivel, y escriben lecciones en su diario que después se inyectan como few-shot en su próximo prompt (`web/lib/progression.ts`). Esto es lo que hace al sistema **genético**: el aprendizaje se hereda entre generaciones de builds.

Sobre 1 GPU (RunPod A40 48GB), los 75 agentes **no** corren como 75 procesos simultáneos — son 75 definiciones reales que se ejecutan sobre la cola compartida de Ollama. El propio archivo es honesto sobre esto: *"they are 75 real definitions, not 75 simultaneous processes — that's a hardware limit, not a design one."* El mismo roster escala a workers paralelos + un grafo cuando el hardware lo permite.

2. Team Structure

El roster se divide en **3 equipos (islas genéticas)**, definidos en `TEAMS` (`web/lib/roster.ts`). Cada equipo corre una **filosofía de desarrollo distinta** (un "lens" inyectado en el system prompt) **y un modelo distinto** (diversidad de modelo + filosofía para un ensemble real). Esto está en `TEAM_MODEL` (`web/lib/tournament.ts`).

Team	id	Philosophy	Label (UI)	Modelo por defecto	Stat-mod (<code>PHIL_MOD</code>)
Lean	A	lean	Equipo Lean	qwen2.5-coder (<code>MODELS.code</code>)	speed +2, creativity +1, rigor -1, depth -1
Enterprise	B	ddd	Equipo Enterprise	deepseek-coder-v2	depth +2, rigor +2, speed -2, creativity -1
Hacker	C	hacker	Equipo Hacker	devstral	creativity +2, speed +1, security +1, rigor -2

El `PHIL_MOD` se aplica sobre los stats base de cada rol (`applyMod()`), así que **el mismo rol tiene stats — y por ende temperatura — distintos según el equipo**. Un Backend Engineer en el equipo Lean corre a $T \approx 0.37$; el mismo rol en Enterprise (DDD) corre a $T \approx 0.15$; en Hacker a $T \approx 0.45$. La filosofía del equipo literalmente reconfigura cómo piensa cada agente.

2.1 Lean Team (Team A)

- **Objetivo:** la cosa más simple que funciona de punta a punta. YAGNI.
- **Estrategia / lens (verbatim de `TEAMS`):** *"Ship the SIMPLEST thing that fully works. YAGNI: fewer tables, fewer files, no speculative abstraction. Prefer obvious, boring, correct code over clever code."*
- **Modelo:** qwen2.5-coder (el `MODELS.code` por defecto local — qwen2.5-coder:7b).
- **Carácter:** speed alto, rigor recortado. Tiende a blueprints con menos tablas y rutas finas.

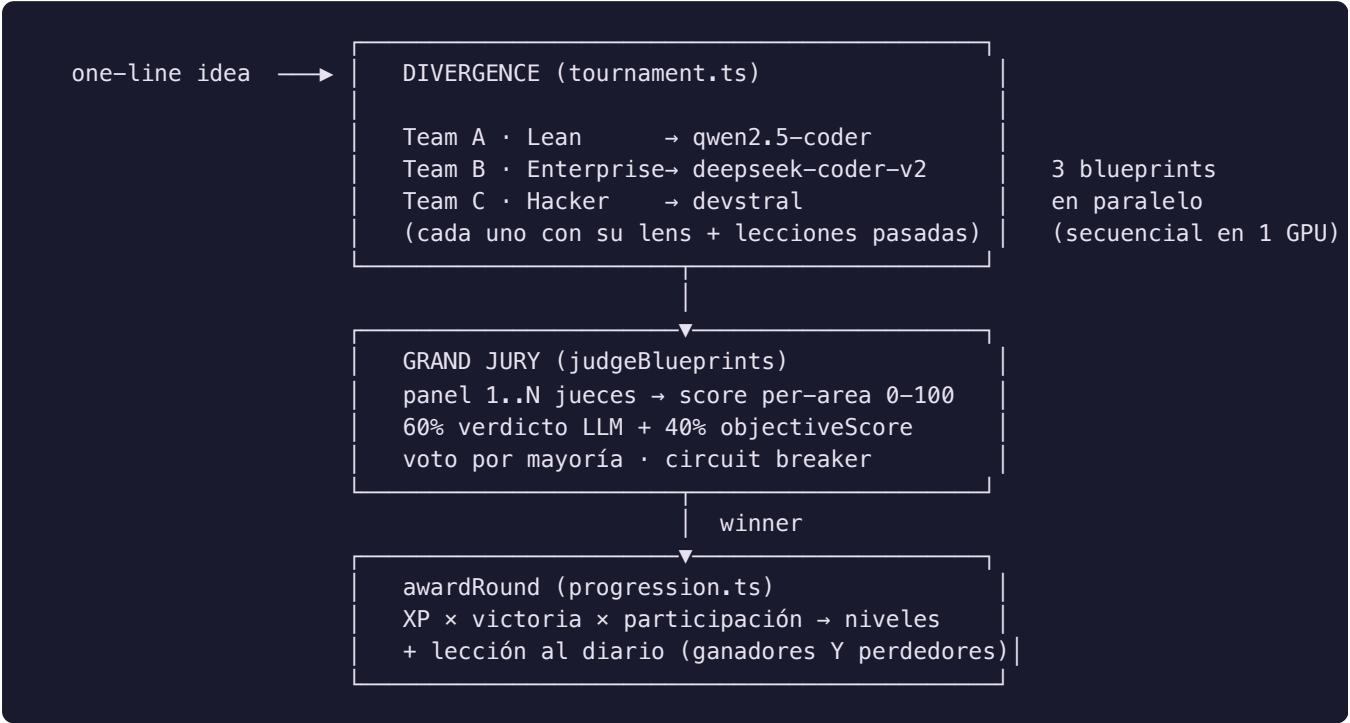
2.2 Enterprise Team (Team B)

- **Objetivo:** modelo de dominio rico, por capas, con contratos explícitos y validación defensiva.
- **Estrategia / lens (verbatim):** *"Model the domain richly and explicitly; clear layering and contracts; validate inputs defensively; correctness and maintainability over raw speed."*
- **Modelo:** deepseek-coder-v2 .
- **Carácter:** depth +2 y rigor +2, speed -2 → corre **frío** (más determinista). Produce los blueprints más profundos y suele ganar en el área `data` .

2.3 Hacker Team (Team C)

- **Objetivo:** performance e ingenio; terso, veloz, no convencional, con ojo de seguridad ofensiva.

- **Estrategia / lens (verbatim):** "Be clever and terse; optimize the hot path; question assumptions; probe security/edge-cases aggressively. Unconventional but it must still compile and run."
- **Modelo:** `devstral`.
- **Carácter:** creativity +2 y security +1, rigor -2 → corre **caliente**. Es el equipo que rompe supuestos y encuentra edge-cases adversariales.



3. Roles

Hay **25 roles base** (`BASE` en `web/lib/roster.ts`), cada uno con stats RPG base, una `room` del campus 2.5D, y un `persona` seed que se usa para el sprite. Multiplicado por los 3 equipos da los **75 agentes**. Cada rol pertenece a un **área** (`ROLE_AREA` en `web/lib/progression.ts`): `data` , `dev` , `design` o `business` — eso determina qué playbook recibe y qué score de área cobra en el torneo.

Roster completo, agrupado por área, con los **stats base reales** (`creativity` , `rigor` , `security` , `speed` , `depth`) y la temperatura base derivada (la columna T es la del rol base; cada equipo la desplaza con `PHIL_MOD`):

Área `data` (arquitectura/datos → playbook `ARCHITECT`)

Rol	name	Stats base (c,r,sec,sp,d)	T base	Room
<code>master-spec-architect</code>	Arquitecto de Spec	7,8,4,5,9	0.37	ti
<code>domain-architect</code>	Arq. de Dominio	7,8,5,5,9	0.37	ti
<code>data-architect</code>	Arq. de Datos	6,8,5,5,9	0.31	ti
<code>contracts-architect</code>	Arq. de Contratos	5,8,5,5,8	0.24	ti
<code>data-engineer</code>	Data Engineer	5,8,5,6,7	—	ti

Área **dev** (desarrollo → playbook **DEV**)

Rol	name	Stats base (c,r,sec,sp,d)	T base	Room
backend-engineer	Backend Engineer	5,7,6,6,7	0.27	ti
reliability-engineer	QA / Confiabilidad	4,10,7,5,6	0.13	ti
completeness-critic	Crítico de Completitud	6,9,5,5,7	0.28	ti
reliability-verifier	Runtime Verifier	4,9,6,6,6	—	ti
security-engineer	Ing. de Seguridad	5,9,10,5,7	0.22	ti
ci-fixer	CI Fixer	5,9,5,7,6	—	ti
devops	DevOps	5,7,6,8,6	—	ti

Área **design** (diseño/UX → playbook **DESIGN**)

Rol	name	Stats base (c,r,sec,sp,d)	T base	Room
art-director	Director de Marca	10,3,2,5,5	0.67	design
frontend-engineer	Frontend Engineer	8,5,4,6,6	0.51	design
frontend-architect	Arq. Frontend	6,7,4,5,8	0.33	design
design-qc	Design QC	6,8,4,5,5	—	design

Área **business** (negocio/growth → playbook **REVIEW**)

Rol	name	Stats base (c,r,sec,sp,d)	Room
discovery-interviewer	Entrevistador	7,6,3,7,6	business
answer-extractor	Extractor	4,7,3,8,5	business
researcher	Researcher	8,6,4,6,7	business
reference-studier	Reference Studier	7,7,4,5,7	business
analyst	Analytics	5,7,4,6,6	business
seo-specialist	SEO	7,6,4,6,5	business
tech-writer	Technical Writer	6,7,3,6,6	business
business-strategist	Business Strategist	7,6,4,6,7	business

Management

Rol	name	Stats base (c,r,sec,sp,d)	T base	Room	Flag
queen-bee	Abeja Reina	8,8,6,6,9	0.43	management	queen: true

A continuación los roles que el documento pide detallar (responsabilidades · inputs · outputs), mapeados a su lugar real en el pipeline (`web/lib/app-builder.ts`).

3.1 Architect (`domain-architect` / `master-spec-architect` — área `data`)

- **Stats reales** (`domain-architect` base): creativity 7, rigor 8, security 5, speed 5, **depth 9** — el depth más alto del roster junto al spec architect y la Reina. Recibe el playbook `ARCHITECT` (*spec-driven, design spec-FIRST not table-first*).
- **Responsabilidad:** diseñar el **blueprint funcional completo** del producto — tablas, operaciones de API y páginas de UI — vía `planBlueprint()`. Es el agente que efectivamente **diverge** en el torneo: corre una vez por equipo, con el `lens` de su equipo inyectado.
- **Inputs:** `DomainConfig` (idea + entidades hint), `contracts` (el contrato canónico de nombres), `reference` (la barra de fidelidad del producto clonado, si aplica), el `lens` del equipo, y `opts.lessons` (las lecciones relevantes pasadas, vía `relevantLessons()`).
- **Outputs:** un `Blueprint` JSON: `{ kind: "public"|"accounts", summary, tables[], routes[], pages[], nav[] }`, normalizado y deduplicado.

3.2 Frontend (`frontend-engineer` + `art-director` + `frontend-architect` + `design-qc` — área `design`)

- **Art Director** — stats `10,3,2,5,5`: la **creativity máxima (10)** y el rigor mínimo (3) del roster → T base 0.67 (la más caliente). **Input:** producto + paleta + screens. **Output:** un brief visual decisivo (la única fuente de verdad que el frontend sigue verbatim).
- **Frontend Engineer** — stats `8,5,4,6,6` → T base 0.51. **Responsabilidad:** escribir **una** página Next.js 16 (App Router) real, interactiva y premium por vez, que compile bajo `tsc --noEmit`. Recibe el playbook `DESIGN` vía `skillFor("design")`. **Inputs:** archivo, ruta, título, behavior exacto, nav, exemplar. **Output:** el `.tsx` de la página.
- **Frontend Architect** — stats `6,7,4,5,8`: escribe el **shell** `app/app/layout.tsx`, bespoke al producto (no un layout genérico centrado).
- **Design QC** — stats `6,8,4,5,5`: revisor implacable; devuelve **solo** defectos visuales/UX críticos con su fix code-level.

3.3 Backend (`backend-engineer` — área `dev`)

- **Stats reales:** creativity 5, rigor 7, **security 6**, speed 6, depth 7 → T base 0.27 (corre frío, como corresponde a código). Recibe el playbook `DEV` vía `skillFor("dev")` (*"First-class code: correct, tested, ZERO hardcoding"*).
- **Responsabilidad:** escribir **un** route handler Next.js 16 por vez, implementando **todos** los métodos HTTP listados con lógica real (no TODOs, no stubs), compilable bajo `tsc --noEmit`.
- **Inputs:** path del archivo, métodos a implementar, las operaciones (SQL + reglas) y un `exemplar` (un ejemplo verificado de la librería de exemplars).
- **Output:** el `route.ts` con SQL parametrizado, lógica de dominio importada de `lib/`, status codes correctos y manejo de errores explícito.

3.4 QA (`reliability-engineer` + `reliability-verifier` + `completeness-critic` + `ci-fixer` — área `dev`)

- **Reliability Engineer** — stats `4,10,7,5,6`: **rigor 10**, el más alto del roster, security 7. T base 0.13 — el agente más **determinista** del sistema. Recibe el playbook `TEST` (*"Tests are PROOF, not*

decoration. Arrange-Act-Assert"). **Responsabilidad:** escribir el test de vitest que prueba la lógica de dominio pura; y, como Reliability Engineer en el repair loop, arreglar bugs runtime-fatales que `tsc` no atrapa, mínimamente. **Input:** helpers de dominio puros / un archivo de ruta. **Output:** `lib/__tests__/domain.test.ts` / el archivo arreglado.

- **Completeness Critic** — stats `6,9,5,5,7` : encuentra cada lugar donde un user-journey real se rompe por algo faltante y devuelve **solo** las piezas que faltan.
- **Runtime Verifier** (`reliability-verifier`, `4,9,6,6,6`) y **CI Fixer** (`5,9,5,7,6`) cierran el gate de runtime y CI.

3.5 Product (`business-strategist` + `discovery-interviewer` + `researcher` + `analyst` — área `business`)

- **Business Strategist** — stats `7,6,4,6,7` : recibe el playbook `REVIEW` (juzga correctness, completitud de journey, naming, patrones, tests, seguridad, over-engineering). Es la voz del "Stakeholder" que también modela el panel del jurado.
- **Discovery Interviewer** (`7,6,3,7,6`) y **Answer Extractor** (`4,7,3,8,5`) hacen el discovery → extracción estructurada.
- **Researcher** (`8,6,4,6,7`) determina si el producto depende de datos externos/live y nombra la fuente real; **Reference Studier** (`7,7,4,5,7`) reconstruye la barra de fidelidad del producto clonado.

3.6 Queen Bee (orquestración — `management`)

- **Stats:** `8,8,6,6,9` con `queen: true` . Recibe el playbook `QUEEN` (*orchestration: plan the whole roadmap as verifiable steps*). En el reparto de XP (`progression.ts`) la Reina es especial: cobra **el mejor área del equipo** (`Math.max` sobre las 4 áreas) con participación 1.0 fija — porque es dueña del deliverable completo.

4. Prompt Construction

Cada llamada a un agente ensambla su prompt de forma **role-scoped**: no hay un "mega-prompt" global; se compone el system prompt del rol + su playbook + el contexto inyectado. El patrón canónico está en `planBlueprint()` y en cada paso de `app-builder.ts` .

4.1 System Prompt

El system prompt arranca con la identidad del rol ("You are the Domain Architect for an app generator...", "You are a Backend Engineer...", "You are a QA engineer..."). Es **específico del rol** y concreto sobre el output esperado (formato JSON estricto, "no TODOs, no stubs", "must compile under `tsc --noEmit` ").

4.2 Context Injection — el playbook

Inmediatamente después de la identidad, se inyecta el **playbook de la disciplina** del agente vía `skillFor(area)` (`web/lib/skill-evolution.ts`). Los playbooks (`web/lib/playbooks.ts`) son metodologías compactas destiladas del ecosistema de skills (Superpowers, frontend-design de Anthropic, addyosmani/agent-skills) — deliberadamente cortas para que un modelo local de 7-32B realmente las siga. El mapeo área → playbook (`BY_AREA`):

Área	Playbook	Esencia
data	ARCHITECT	spec-driven; catalog vs user-generated; contrato explícito; ONE name per concept; money en integer cents
dev	DEV	NO HARDCODING; lógica de dominio en <code>lib/</code> ; rutas finas; SQL parametrizado; código mínimo
design	DESIGN	jerarquía visual; spacing rhythm; contenido real (no lorem); a11y; estados loading/empty/error
business	REVIEW	review crítico: correctness, journey, naming, patterns, tests, seguridad, over-engineering

Hay además dos playbooks **no mapeados por área** que se inyectan en puntos específicos: `TEST` (usado por el QA vía `skillFor("test")`) y `ADVERSARIAL` (inyectado explícitamente en el system prompt del jurado — ver `judgeOnce` en `tournament.ts`, que concatena `${skillFor("business")}\n\n${PLAYBOOK.adversarial}`).

`skillFor()` **no devuelve siempre el playbook estático**. Es el punto de inyección de la evolución de skills (SkillOpt): si hay un skill doc evolucionado y **validado** en la base (`puglit_skills WHERE status='active'`), `skillFor()` devuelve **ese** doc; si no, cae al seed playbook. La firma real:

```
// skill-evolution.ts
export function skillFor(area: SkillArea): string {
  return active[area]?.doc || SEED[area] // SEED = los playbooks de playbooks.ts
}
```

Así, el conocimiento de cada disciplina es **estado entrenable**: un optimizer propone ediciones acotadas y solo se aceptan si superan un set de validación held-out (5 tareas frozen → blueprint → `objectiveScore`). El overlay se carga una vez por build (`loadActiveSkills()`) → costo de inferencia extra cero en generación.

4.3 Module Context

Cuando el agente que diverge diseña el blueprint, se le inyecta el **catálogo de módulos reutilizables** vía `moduleCatalog()`:

"REUSABLE MODULES already in the factory (if the product needs one of these, REUSE it — do NOT design it from scratch; just note it in the blueprint)"

Es el registro vivo de ~85 módulos (`web/lib/module-registry.ts`) que los agentes **ven** y pueden referenciar. La inyección determinística real de esos módulos (los `deterministicX(config, bp) → {files, extraSql}`, p.ej. `deterministicRentals`, `deterministicRag`, `deterministicWallet`) ocurre en **finalize**, no por el LLM — el agente solo se entera de que existen para no reinventarlos.

4.4 Memory Context — las lecciones del diario

El último bloque inyectado son las **lecciones del equipo**, vía `relevantLessons(team, taskText)` (`web/lib/progression.ts`), bajo el encabezado:

```
"LESSONS FROM YOUR TEAM'S PAST PROJECTS (apply them — this is how you improve and beat the other teams)"
```

Esto es la **herencia genética**: cada round escribe lecciones al `puglit_agent_diary` (ganadores registran qué funcionó, perdedores la crítica a mejorar), embebidas con `nomic-embed`. `relevantLessons` no trae las recientes a ciegas — está endurecido contra el envenenamiento de memoria:

- **Relevance floor 0.35**: una lección de otro dominio (fintech → salud) no se puede forzar.
- **Recency decay** (half-life ~31 días): lecciones nuevas pesan más que viejas → sin consejos contradictorios de versiones viejas.
- **Anti-poisoning**: solo recuerda rounds de calidad decente (`quality >= 45`) y **nunca** uno con `outcome = 'failure'`.
- **Devuelve vacío** si nada supera el floor: *"nothing relevant enough → no advice beats wrong advice."*

SYSTEM PROMPT del agente =

- | | |
|---|--------------------|
| 1. Identidad del rol ("You are the Domain Architect...") | |
| 2. <code>skillFor(area)</code> → playbook (o skill evolucionado validado) | |
| 3. <code>moduleCatalog()</code> → módulos reutilizables | (solo divergence) |
| 4. lens del equipo (Lean/DDD/Hacker) | (solo divergence) |
| 5. <code>relevantLessons(team, task)</code> → diario | (memoria genética) |
| 6. reglas + contrato JSON estricto de salida | |

USER PROMPT = producto + pitch + entidades + contracts (+ reference)

5. Context Window Strategy

Los modelos locales (7-32B) tienen ventanas chicas comparadas con los frontier. La estrategia es **inyectar lo necesario y nada más**, y delegar el volumen a estado determinístico fuera del prompt.

5.1 Qué entra

- El **playbook compacto** del rol (cortos a propósito: *"Kept short on purpose so a local 7-32B model actually follows it"*).
- Las **lecciones relevantes** (filtradas por relevancia + recencia + calidad), no el diario entero.
- El **contrato canónico** de nombres (tablas/columnas/operaciones) — la única fuente de verdad de naming.
- Para el juez: las **cards anonimizadas** de cada blueprint (Option 1/2/3) — kind, summary, nombres de tablas, paths de rutas, rutas de páginas (sin los archivos completos).
- En revisión de páginas: el código truncado (`code.slice(0, 16_000)`) — explícitamente acotado.

5.2 Qué NO entra

- **El roster completo de 75 agentes:** el jurado puntúa **por equipo y por área** (3 equipos × 4 áreas), nunca los 75 agentes uno por uno. El comentario de `progression.ts` lo dice: *"The Grand Jury scores each TEAM's deliverable PER AREA (1-100) — never the 75 agents one by one (token-cheap)."*
- **Los archivos de los módulos:** se inyecta el catálogo (nombres + qué hacen), no el código. La inserción real es determinística en `finalize`.
- **El diario completo / lecciones de baja calidad / de otro dominio:** filtradas por `relevantLessons`.
- **El código de otros candidatos al juzgar:** cada blueprint se juzga como un todo, *"never mix pieces across candidates."*

6. Agent Communication

6.1 Shared context

Los agentes comparten estado a través de **Postgres**, no de mensajes directos. El contrato canónico de nombres, el blueprint, el catálogo de módulos y el diario son todos artefactos persistidos que el siguiente agente lee. El playbook `ARCHITECT` lo formaliza: *"Emit an EXPLICIT CONTRACT every other agent must follow VERBATIM... ONE name per concept... the dev and the lib helpers must reuse these EXACT names."* La coordinación es por **contrato compartido**, no por chat.

6.2 Isolation

Cada equipo es una **isla genética**: diverge de forma independiente, con su propio modelo, su propio lens y **sus propias lecciones** (`relevantLessons(t.id, ...)` está scopeado por `team`). Un equipo nunca ve el blueprint de otro durante la divergencia. El jurado los recibe **anonimizados** (Option 1/2/3) — *"Judge only on merit, ignore house style"* — para que el voto sea por mérito y no por reconocer un estilo. En el reparto de XP, cada agente cobra solo el score de **su** área (`ROLE_AREA[a.role]`), aislando la señal.

6.3 Voting

El **Grand Jury** (`judgeBlueprints` en `tournament.ts`) es un panel de 1..N jueces (configurable con `PUGLIT_JURY_MODELS` para un "triumvirato" multi-modelo; default: un único `MODELS.premium`). Mecánica de voto:

1. Cada juez puntúa cada candidato **anonimizado** en 4 áreas (0-100) + una crítica de una frase + un ganador, con rúbrica explícita (90-100 ships as-is; <50 broken) y postura **adversarial**.
2. Los scores por área se **promedian** entre jueces.
3. El score final de cada área mezcla **60% veredicto LLM + 40% objectiveScore** (un score determinístico y medible del blueprint) — *"so selection isn't purely subjective."*
4. El **ganador sale por mayoría de votos**; empate → mayor overall agregado (el desempate).
5. **Inter-judge agreement:** se registra la fracción de jueces que votó al ganador como métrica de ruido de la señal de fitness (`recordMetric("judge_agreement", ...)`).

6. **Circuit breaker:** si **todo** el jurado falla, no se bloquea el pipeline — degrada a "draft mode" y elige el blueprint más completo (más rutas + tablas), con score 60 plano.
-

7. Cost Analysis

Puglit corre **100% local en Ollama por defecto** (web/lib/openai.ts): sin API key, **costo de inferencia \$0**. El "costo" real es de **tokens y latencia sobre 1 GPU**, no de dólares — y el sistema está diseñado en torno a esa restricción.

7.1 Tokens

- **Tiers de modelo** (`MODELS`): `premium` (arquitectura/juicio/review), `balanced`, `cheap` (extracción/normalización) y `code` (rutas/páginas/repair). No todo agente usa el mismo cerebro: defaults locales `gemma2:27b` / `gemma2` / `gemma2:2b` / `qwen2.5-coder:7b`. Esto pone el modelo caro **solo** donde la calidad de razonamiento decide.
- **Una sola garganta:** todas las llamadas pasan por `call()` (chokepoint único), con dos protocolos (`openai` + `anthropic`). Cada llamada se traza vía `traceCall` para el scorecard de costo.
- **Puntuación por equipo/área, no por agente:** el jurado emite 3×4 scores en lugar de 75 — la decisión de diseño explícitamente "token-cheap" del sistema.
- **Playbooks cortos + lecciones filtradas + cards anonimizadas:** todo el prompt-building está optimizado para no saturar la ventana de un modelo local.
- **SkillOpt corre OFFLINE:** la evolución de skills nunca corre inline por build; el artefacto evolucionado se sirve con **costo de inferencia extra cero** en generación.

7.2 Latencia

- **La A40 es el cuello de botella, no la red.** Sobre 1 GPU los 75 agentes comparten la cola de Ollama; los 3 equipos y sus modelos divergen **secuencialmente** ("On 1 GPU the 3 teams + models run SEQUENTIALLY (shared queue)"). Cambiar de modelo entre Qwen/DeepSeek/Devstral implica swaps de modelo en la GPU — de ahí que el jurado multi-modelo también corra secuencial ("sequential: 1 GPU swaps").
 - **Temperatura como palanca de latencia/calidad:** los roles deterministas (Reliability Engineer $T \approx 0.13$) convergen rápido; los creativos (Art Director $T \approx 0.67$) exploran más.
 - **Degradación graceful:** si el modelo de un equipo no está pulleado o devuelve output imparseable, cae al `MODELS.code` por defecto para que el equipo **siga compitiendo** en lugar de abortar el round.
 - **El swarm profiler** (web/lib/swarm-profile.ts + `run-trace.ts`) puntúa cada build en Cost/Latency/Reliability/Context y emite fixes rankeados.
-

8. Failure Modes

El sistema asume que los modelos locales fallan seguido y está construido para **degradar, no romper**. Modos de falla reales y sus mitigaciones, verificados en el código:

Modo de falla	Dónde	Mitigación
Modelo de un equipo no pulleado / output imparseable	<code>divergeBlueprints</code>	Fallback a <code>MODELS.code</code> con etiqueta "(fallback de X)"; si el fallback también falla, el equipo se saltea pero los otros compiten.
Jurado entero falla	<code>judgeBlueprints</code>	Circuit breaker → "draft mode": gana el blueprint más completo, score 60 plano, métrica <code>judge_failed</code> .
Jurado ruidoso / poco fiable	<code>judgeBlueprints</code>	Mezcla 60% LLM + 40% <code>objectiveScore</code> (determinístico) + métrica de inter-judge agreement.
Memory poisoning (lección mala que se hereda)	<code>relevantLessons</code>	Relevance floor 0.35 + recency decay + <code>quality>=45</code> + <code>outcome<>'failure'</code> ; devuelve vacío antes que ruido.
Skill evolucionado que sobreajusta	<code>skill-evolution.ts</code>	Solo se acepta una edición si supera un set de validación held-out (5 tareas frozen); rejected-edit buffer + edit budget.
Hardcoding / SQLi / phantom-table	<code>swarm-checks.ts</code> / <code>swarm-repair.ts</code>	Scan estático de seguridad/consistencia + repair de tablas fantasma (infiere schema de la intención) + escalada a frontier.
Confident-but-wrong (un answer confiado no es correcto)	<code>adversarial-review.ts</code>	3 lentes ortogonales (Auditor/Adversary/Pragmatist); un hallazgo confirmado por ≥2 lentes es real → SHIP / SHIP-WITH-CAVEATS / BLOCK.
Compila verde pero rompe en runtime	<code>build-local.mjs</code>	Runtime gate: bootea la app, smoke-testea páginas (0 5xx), corre vitest + coverage + Queen evidence review que rebota si está bajo la barra. En el litmus test Stayforge este gate expuso 7 bugs reales invisibles a un compile verde.
Naming drift (tabla "reservations" en schema, "bookings" en ruta)	playbooks <code>ARCHITECT</code> / <code>DEV</code> / <code>REVIEW</code>	Contrato canónico VERBATIM: ONE name per concept; el review lo flaggea como fail.
Tie en la votación del jurado	<code>judgeBlueprints</code>	Desempate por mayor overall agregado, luego por <code>designs[0]</code> .
Pérdida de identidad/memoria entre runs	<code>progression.ts</code> + <code>brain-sync.ts</code>	Estado en Postgres (append-only CRDT grow-set) + mirror a Obsidian + snapshots a git/bucket; skills mutables arbitrados por <code>val_score</code> .

La filosofía transversal de fallas es la misma del playbook `ADVERSARIAL`: "*a confident answer is not a correct one.*" En cada punto donde un LLM podría estar seguro y equivocado — el arquitecto, el jurado, la memoria, el skill, el código — hay un gate determinístico que verifica contra evidencia real (el código, los tests, el coverage, el runtime) en lugar de confiar en la afirmación.